Indian rulers are listed in a relation:

Rulers(name, epithet, dynasty, beginReign, endReign)

- **name** is unique. It may include a regnal number (e.g., *"Akbar I"*, *"Shivaji II"*) or an identifying tag that distinguishes rulers with the same common name (e.g., *"Harsha of Kannauj"*).
- **epithet** is an additional title/sobriquet (e.g., *"The Great"*, *"The Tiger of Mysore"*, *"Rajadhiraja"*) **only if it is not already part of name**.
- epithet is **NULL** if the ruler has no such epithet recorded.
- **dynasty** is the ruling house/lineage (e.g., *"Mughal"*, *"Maurya"*, *"Chola"*, *"Maratha"*).
- **beginReign** and **endReign** are integers giving the first and last years (inclusive) during which the ruler held power.

There is also a relation:

Parentage(child, parent)

Both attributes are names from Rulers. A tuple (c, p) means **c is a child of p**.

RoyalHeirs(child) — the set of "notable heirs" (or designated princes/princesses) that we care about.

Answer the following SQL questions.

Find the names of rulers who were the parent of two or more rulers. List each such person only once. 1/1

- [x] SELECT parent FROM Parentage GROUP BY parent HAVING COUNT(*) >= 2;
- [x] SELECT DISTINCT p.parent FROM Parentage p WHERE (SELECT COUNT(*) FROM Parentage q WHERE q.parent = p.parent) >= 2;
- [x] SELECT DISTINCT p1.parent FROM Parentage p1 WHERE EXISTS (SELECT 1 FROM Parentage p2 WHERE p2.parent = p1.parent AND p2.child <> p1.child);
- [x] SELECT DISTINCT p1.parent FROM Parentage p1 JOIN Parentage p2 ON p1.parent = p2.parent AND p1.child <> p2.child;

Return the **name** and **epithet** of the ruler(s) whose reign includes the year 1000. 1/1

- [x] SELECT name, epithet FROM Rulers WHERE 1000 BETWEEN beginReign AND endReign;
- [x] SELECT name, epithet FROM Rulers WHERE beginReign <= 1000 AND endReign >= 1000;
- [ ] SELECT name, epithet FROM Rulers WHERE beginReign <= 1000 OR endReign >= 1000;
- [ ] SELECT name, epithet FROM Rulers WHERE beginReign < 1000 AND endReign > 1000;
- [x] SELECT name, epithet FROM Rulers WHERE beginReign <= 1000 AND endReign >= 1000 UNION SELECT name, epithet FROM Rulers WHERE 1000 BETWEEN beginReign AND endReign;

Find parents who are not the parent of any child listed in the **RoyalHeirs** set. 1/1

- [x] SELECT DISTINCT p.parent FROM Parentage p WHERE NOT EXISTS (SELECT 1 FROM Parentage p2 JOIN RoyalHeirs h ON p2.child = h.child WHERE p2.parent = p.parent);
- [x] SELECT parent FROM Parentage EXCEPT SELECT DISTINCT p2.parent FROM Parentage p2 JOIN RoyalHeirs h ON p2.child = h.child;
- [ ] SELECT DISTINCT p.parent FROM Parentage p WHERE EXISTS (SELECT 1 FROM Parentage p2 JOIN RoyalHeirs h ON p2.child = h.child WHERE p2.parent = p.parent);
- [ ] SELECT parent FROM Parentage EXCEPT SELECT DISTINCT p2.child FROM Parentage p2 JOIN RoyalHeirs h ON p2.child = h.child;
- [x] SELECT DISTINCT p.parent FROM Parentage p LEFT JOIN (SELECT DISTINCT p2.parent FROM Parentage p2 JOIN RoyalHeirs h ON h.child = p2.child) ph ON ph.parent = p.parent WHERE ph.parent IS NULL;

Find the name and epithet of all rulers whose epithet is not NULL and does not start with "The". 1/1

- [x] SELECT name, epithet FROM Rulers WHERE epithet IS NOT NULL AND epithet NOT LIKE 'The%';
- [ ] SELECT name, epithet FROM Rulers WHERE epithet <> NULL AND epithet NOT LIKE 'The%';
- [x] SELECT name, epithet FROM Rulers WHERE epithet NOT LIKE 'The%';
- [ ] SELECT name, epithet FROM Rulers WHERE epithet IS NOT NULL OR epithet NOT LIKE 'The%';

Find all pairs of rulers **(A, B)** such that **A is the great-grandchild of B**. 1/1

- [x] SELECT P1.child AS A, P3.parent AS B FROM Parentage P1 JOIN Parentage P2 ON P1.parent = P2.child JOIN Parentage P3 ON P2.parent = P3.child;
- [x] SELECT p1.child AS A, p3.parent AS B FROM Parentage p1, Parentage p3 WHERE EXISTS (SELECT 1 FROM Parentage p2 WHERE p1.parent = p2.child AND p2.parent = p3.child);
- [x] SELECT p1.child AS A, p3.parent AS B FROM Parentage p1 JOIN Parentage p3 ON p1.parent IN (SELECT p2.child FROM Parentage p2 WHERE p2.parent = p3.child);
- [ ] SELECT P1.child AS A, P3.parent AS B FROM Parentage P1 JOIN Parentage P2 ON P1.parent = P2.parent JOIN Parentage P3 ON P2.child = P3.child;
- [ ] SELECT p1.child AS A, p3.parent AS B FROM Parentage p1, Parentage p3 WHERE EXISTS (SELECT 1 FROM Parentage p2 WHERE p1.parent = p2.child OR p2.parent = p3.child);